MSc in Financial Engineering

ADVANCED MACHINE LEARNING AND FINANCE

GROUP ASSIGNMENT

GROUP 6

85750 – Jade BLANC
85769 – Marie CALDEIREIRO
85878 – Anas LAMRI

Project Title:

From Naive Bayes to Kernel Methods: Financial Risk Modeling

Lecturer: Nicolas LOPEZ

December 1, 2025

Contents

1 Task A — Credit Default Prediction (Classification) 2

1.1 A.1 Model Construction . . . 2

1.1.1 Learned Results . . . 2

1.1.2 Example of a Prediction . . . 2

1.2 A.2 Parameter Estimation . . . 2

1.3 A.3 Default Probability . . . 3

2 Task B — Asset Return Modeling Using Kernel Methods 4

2.1 B.1 Data Simulation and Problem Setup . . . 4

2.2 B.2 Linear vs. Nonlinear Modeling . . . 5

2.3 B.3 Gram Matrix Construction . . . 7

2.4 B.4 Prediction with the Kernel Model . . . 8

2.5 B.5 Model Comparison and Financial Interpretation . . . 9

2.6 Conclusion . . . 11

List of Figures

1 Actual vs. Predicted Returns by Regime. The dashed line represents a perfect fit, while points correspond to predictions in low- and high-volatility regimes. . . 5

2 Example of Gram Matrix Output, Including Matrix Statistics and a 5x5 Block of Entries. . . . 7

3 Training set: actual vs. predicted returns for the linear model (left) and the kernel regression model (right). The kernel model adheres more closely to the 45° line, especially in nonlinear high-volatility regions. . . . 10

4 Test set: actual vs. predicted returns for the linear and kernel models. The kernel model forms a tighter cloud around the diagonal, although both models exhibit prediction noise due to the variability in the test sample. . . . 11

List of Tables

1 Extract of Model Parameters and Example Probability Output . . . 4

2 OLS Regression Summary for Predicting Asset Returns . . . 6

3 Kernel regression: first five training predictions . . . 8

4 Test-set predictions: kernel vs. linear regression . . . 9

5 Prediction Error (MSE) for Linear vs. Kernel Regression . . . 9

1 Task A — Credit Default Prediction (Classification)

1.1 A.1 Model Construction

In this first step, we build a Naive Bayes model to predict whether a client is likely to default. The key assumption is that, conditional on the default status (defaulter vs. non-defaulter), the financial features (age, income, debt ratio, past defaults, credit score) are mutually independent. Although unrealistic, this greatly simplifies estimation and yields a fast, interpretable classifier.

Because all features are numerical, we model their distribution within each class by a Gaussian ("bell-shaped") law. For each feature and each class (default / no default), the algorithm only needs to estimate:

- the average value,
- the variability,
- and the class prior probability.

1.1.1 Learned Results

After training the model on 5,000 synthetic clients, the estimated class frequencies are:

- Non-default: 99.46%,
- Default: 0.54%.

This confirms that defaults are rare events, as typically observed in credit datasets.

1.1.2 Example of a Prediction

For a hypothetical client (35 years old, $40k income, debt ratio 0.5, one past default, credit score 620), the model estimates:

- **Probability of no default**: 98.8%,
- **Probability of default**: 1.2%.

Even though this profile is riskier than average, the predicted probability remains low because the model incorporates the very small base rate of default. Overall, Naive Bayes learns distinct behavioural patterns for defaulters and non-defaulters and assigns default probabilities to new clients by comparing their features to these two groups.

1.2 A.2 Parameter Estimation

Proportion of Clients Who Default

The first parameter is the overall share of clients who actually default, i.e. the percentage of observations with `default = 1`. In our dataset, this proportion is approximately **0.54%**, confirming that default events are rare, as in real credit portfolios.

Distribution of Features Within Each Class

For each feature (age, income, debt ratio, past defaults, credit score), we split the data into:

- clients who do **not** default,
- clients who **do** default.

Within each group, we compute:

- the **average** of each feature,
- its **variability**.

Assuming Gaussian distributions, these two quantities fully characterize the behaviour of each feature inside each class and are exactly the parameters used by the Naive Bayes classifier.

Results

The estimated parameters show clear differences between the two groups:

- Defaulters have **lower income**, **higher debt ratios**, **more past defaults** and **lower credit scores**.
- Non-defaulters display the opposite pattern.

These contrasts explain why the model assigns higher default probabilities to riskier profiles.

1.3 A.3 Default Probability

After training the Naive Bayes model, we compute a default probability for each client. These probabilities combine the class priors with the class-conditional feature distributions and represent the likelihood that a client belongs to the "default" group.

Comparison with the True Outcomes

Most clients receive very low predicted default probabilities, which is consistent with the overall default rate of about 0.54%. Clients who actually default tend to have higher predicted values (typically in the 1–5% range), while non-defaulters almost always obtain probabilities close to 0%. This shows that the model is conservative but correctly ranks clients by risk.

Are the Predictions Realistic?

From a credit-risk perspective, these probabilities are reasonable: they reflect the rarity of default events and assign higher risk to clients with low income, high debt ratios, past defaults, and low credit scores. Even for defaulters, the probabilities remain modest because the prior default probability is extremely small. The model therefore avoids extreme predictions while still identifying the riskiest profiles.

Model Parameters and Example Output

Table 1: Extract of Model Parameters and Example Probability Output

Classes: [0, 1]
Priors (P(Y=c)): [0.9946, 0.0054]

Means of each feature by class (theta_):
Class 0: [3.99111472e+01, 4.60087805e+04, 2.82966293e-01, 2.84938669e-01, 6.44808915e+02]
Class 1: [4.14500010e+01, 3.72957228e+04, 3.43606321e-01, 4.44444444e-01, 6.31794977e+02]

Variances of each feature by class (var_):
Class 0: [9.47586133e+01, 6.15213136e+02, 2.55649270e-02, 2.82575282e-01, 6.47560180e+03]
Class 1: [6.76799808e+01, 5.23879373e+02, 2.43556466e-02, 4.69136802e-01, 8.02621606e+03]

Predicted Probabilities:
Probability client does **not** default: 0.9880384616331097
Probability client **defaults**: 0.011961538366890432

2 Task B — Asset Return Modeling Using Kernel Methods

2.1 B.1 Data Simulation and Problem Setup

We simulate daily asset returns over roughly one trading year (260 days) using two market factors: past returns and a volatility indicator.

Market Factors

- **Past Returns**: Normally distributed with mean 5% and standard deviation 15%.
- **Volatility**: Chi-squared distributed (to ensure positivity) and scaled to realistic levels.

Nonlinearity via Regime-Switching

We introduce nonlinear behaviour through volatility regimes:

- **High Volatility**: days where volatility exceeds the median.
- **Low Volatility**: days where volatility is below the median.

Asset Return Simulation

- **Low Volatility Regime**: returns follow an almost linear relationship with past returns and volatility, plus small noise.

- **High Volatility Regime**: returns exhibit nonlinear behaviour with amplified sensitivity to past returns, a quadratic effect of volatility, and an interaction between past returns and volatility. Noise is larger to reflect market uncertainty.

The final dataset contains:

- `Past_Returns`,
- `Volatility`,
- `Asset_Return`,
- `Regime` (high vs. low volatility).

This framework allows us to compare how linear and nonlinear models capture regime-dependent return dynamics.

2.2 B.2 Linear vs. Nonlinear Modeling

First, we fit a **standard linear regression**. This provides our benchmark.

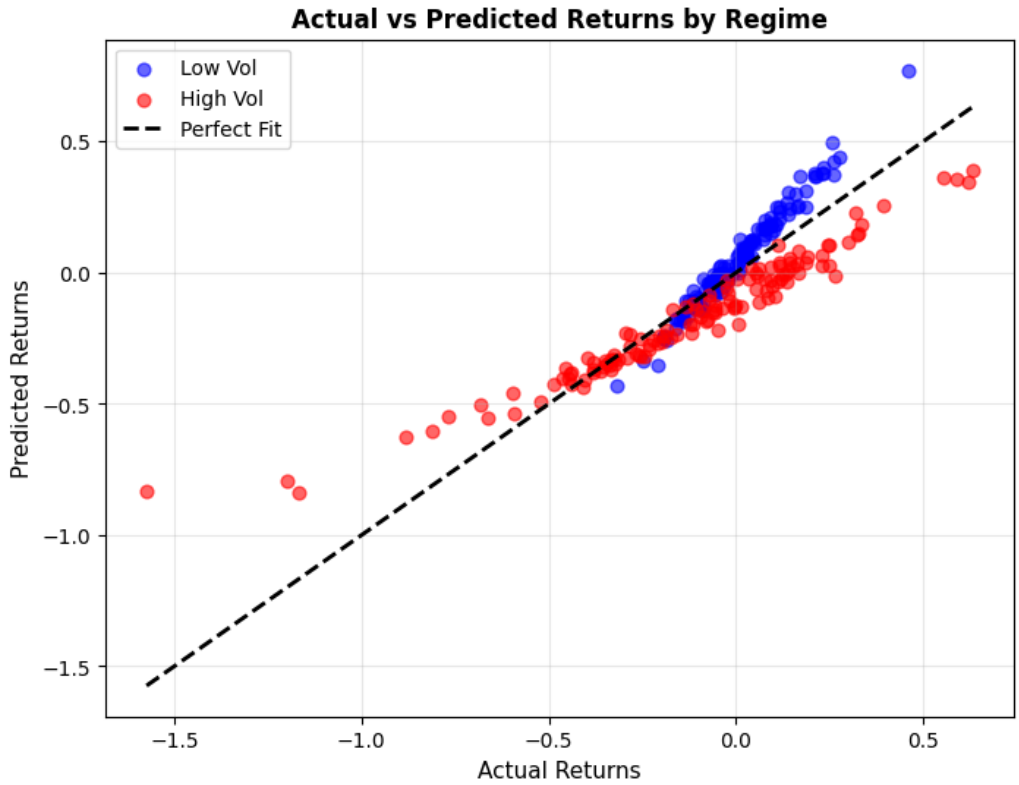

Figure 1: Actual vs. Predicted Returns by Regime. The dashed line represents a perfect fit, while points correspond to predictions in low- and high-volatility regimes.

OLS Regression Results

Below are the results from the fitted linear regression model:

Table 2: OLS Regression Summary for Predicting Asset Returns

Coefficients:				
const:	0.0718	std err: 0.011	t = 6.318	p < 0.001
X1:	1.1928	std err: 0.049	t = 24.475	p < 0.001
X2:	-0.6501	std err: 0.029	t = -22.425	p < 0.001
Residual diagnostics:				
Jarque–Bera (JB): 532.057				
Skew: -1.175	Prob(JB): 2.92×10^{-116}			
Kurtosis: 9.602	Cond. No.: 7.18			
Performance Metrics:				
MSE: 0.01131	R^2: 0.8139	Adjusted R^2: 0.8125		

1. **How well does the linear model fit the data?**

   - The linear regression provides a strong overall fit. The model achieves an R^2 of 0.814 (adjusted $R^2 = 0.812$), meaning it explains around 81% of the variation in asset returns.
   - Both predictors are highly significant: past returns have a strong positive effect (coef = 1.19, $t = 24.48$, $p < 0.001$), and volatility has a strong negative effect (coef = -0.65, $t = -22.43$, $p < 0.001$).
   - The low MSE (0.0131) also indicates good predictive accuracy.
   - The scatter plot in Figure 1 supports this: most points lie close to the perfect-fit line, especially in the low-volatility regime (blue points), where prediction errors are smaller and more symmetric.
   - Overall, the linear model performs well as a **baseline**, but it is not sufficient.

2. **Does the linear model fail to capture the nonlinear pattern introduced?**

   - Yes. Despite the high R^2, the residual diagnostics clearly indicate nonlinearity. The residual distribution fails normality tests: the Omnibus test ($p < 0.001$) and the extremely large Jarque–Bera statistic (JB = 532.1) show strong skewness (-1.18) and heavy tails (kurtosis = 9.60).
   - These patterns suggest systematic structure left in the residuals, meaning the linear model cannot fully capture the nonlinear relationship intentionally built into the data.
   - The graph reinforces this: in the high-volatility regime (red points), predictions consistently fall above or below the perfect-fit line in a curved pattern. This shows that the true relationship bends while the linear model forces a straight line. This systematic deviation across volatility levels confirms that the model does not capture the nonlinear structure.

Then, we built a **kernel regression model** using a degree-2 polynomial kernel, expanding the model's expressive power.

2.3 B.3 Gram Matrix Construction

To model nonlinear relationships, we work in the kernel (dual) space by computing a Gram matrix based on a polynomial kernel.

1. Definition and Kernel Choice

Given training inputs $\{x_i\}_{i=1}^n$, the Gram matrix K is defined by $K_{ij} = k(x_i, x_j)$. We use a degree-2 polynomial kernel, which implicitly introduces both linear and quadratic terms.

2. Computation and Checks

For each pair of training points, we evaluate the kernel and obtain an $n \times n$ symmetric matrix. We then:

- verify symmetry $K_{ij} = K_{ji}$,
- inspect basic statistics (minimum, maximum, mean),
- display a small block of K (Figure 2) to illustrate its structure.

```
Gram Matrix Construction
==================================================
Gram matrix shape: (260, 260)
Gram matrix symmetry check: True

Gram matrix statistics:
  Min value:  0.6332
  Max value:  7.9328
  Mean value: 1.1980

First 5x5 block of Gram matrix:
[[1.04593987 1.04526886 1.06885248 1.08009565 1.02949393]
 [1.04526886 1.10159446 1.09134937 1.04064969 1.06848114]
 [1.06885248 1.09134937 1.11326099 1.10439005 1.06035567]
 [1.08009565 1.04064969 1.10439005 1.16743826 1.0247783 ]
 [1.02949393 1.06848114 1.06035567 1.0247783  1.04632649]]
```

Figure 2: Example of Gram Matrix Output, Including Matrix Statistics and a 5x5 Block of Entries.

Interpretation of the Gram Matrix

The Gram matrix serves as the core component of the kernel regression model, enabling nonlinear relationships to be learned without explicitly transforming the input features. Each element

$K[i,j]$ of the matrix represents the similarity between observation i and observation j in the feature space implicitly defined by the kernel.

Using a degree-2 polynomial kernel, the values capture both:

- the linear contributions of the factors $(x[i]^\top x[j])$,
- the interactions between factors $((x[i]^\top x[j])^2)$.

A high value indicates that two observations are close or behave similarly in the transformed (nonlinear) feature space, while a low value indicates low similarity.

Alpha and Regularization

The previously computed Gram matrix serves as the core of the kernel regression model. A small regularization term ($\lambda = 0.001$) is added to the diagonal to prevent overfitting and ensure stability. The model then solves the linear system:

$$(K + \lambda I)\alpha = y$$

to obtain the coefficients α, which determine the contribution of each training point to the predictions. We also printed basic statistics of α to provide insight into how the training data influences the model.

This setup enables the regression to capture nonlinear relationships while controlling for overfitting.

2.4 B.4 Prediction with the Kernel Model

To complement the linear benchmark, we estimate a kernel regression model using a degree-2 polynomial kernel. After computing the dual coefficients α from the training Gram matrix, we use the model to predict asset returns both for the training sample and for a separate test set composed of new simulated market states.

Training Predictions

The kernel model produces the following predictions for the first five observations of the training dataset:

Table 3: Kernel regression: first five training predictions

Observation	True Return	Kernel Prediction
1	0.078049	0.086988
2	–0.062372	–0.012576
3	0.071013	0.121498
4	0.172435	0.215121
5	–0.012695	–0.015560

These values already indicate that the kernel model captures meaningful nonlinear structure in the data-generating process. Across the full training sample, the kernel regression achieves a substantially lower in-sample error (MSE = 0.002986) compared to the linear benchmark (MSE = 0.012979), consistent with its greater flexibility.

Test Predictions

We then evaluate both models on a fresh test set composed of new simulated market states. For the first five observations, the predictions are:

Table 4: Test-set predictions: kernel vs. linear regression

Obs.	True Return	Kernel Pred.	Linear Pred.
1	–0.047372	–0.285753	–0.323660
2	0.357006	0.128641	0.300947
3	0.332307	0.058385	0.033851
4	–0.062220	–0.250622	–0.294557
5	0.108037	–0.138952	–0.191793

Both models incur larger prediction errors on the test set, which is expected because the test points introduce additional nonlinear patterns and noise. However, the kernel model remains competitive, achieving slightly lower test MSE (0.074609) than the linear regression (0.075617). Overall, the results demonstrate that the kernel regression generalizes reasonably well and adapts to nonlinear return dynamics more effectively than the linear specification.

2.5 B.5 Model Comparison and Financial Interpretation

To evaluate the relative performance of the linear regression benchmark and the degree-2 polynomial kernel regression, we compare their prediction accuracy on both the training sample and the test dataset. We further analyse the visual diagnostic plots and interpret the results from a financial modelling perspective.

Prediction Error Comparison

Table 5 reports the Mean Squared Error (MSE) for both models. The kernel regression achieves a substantially lower in-sample error but only a slightly improved test error.

Table 5: Prediction Error (MSE) for Linear vs. Kernel Regression

Model	Training MSE	Test MSE
Linear Regression	0.012979	0.075617
Kernel Regression (degree-2)	0.002986	0.074609

The kernel model dramatically improves the training fit (MSE reduced by a factor of four), which is consistent with the fact that polynomial kernels implicitly expand the feature space to include nonlinear interactions and quadratic terms. Out-of-sample, the improvement is more modest, reflecting the noise inherent in financial returns and the classical bias–variance trade-off.

Visual Comparison of Predicted vs. Actual Returns

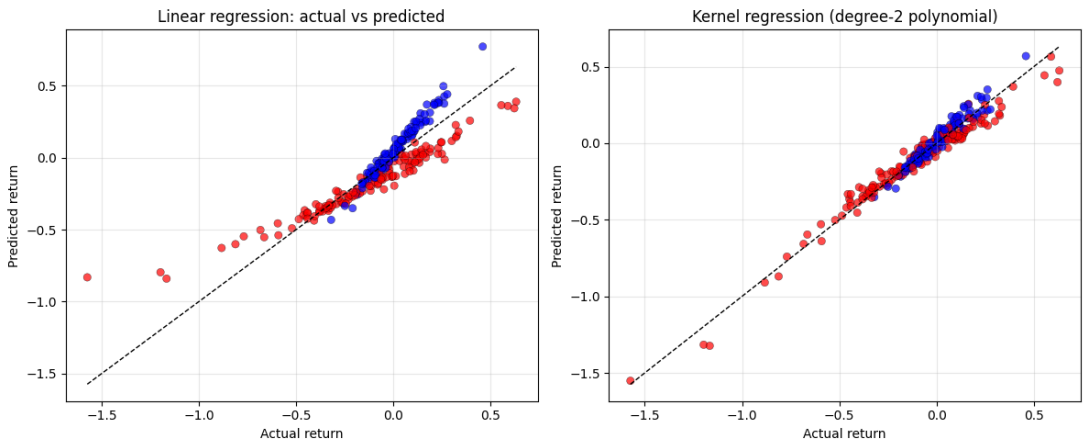

Figure 3: Training set: actual vs. predicted returns for the linear model (left) and the kernel regression model (right). The kernel model adheres more closely to the 45° line, especially in nonlinear high-volatility regions.

In the training set (Figure 3), the linear model displays systematic deviations from the diagonal: it underestimates high positive returns and overestimates negative returns. In contrast, the kernel model successfully captures curvature and nonlinear regime-dependent behaviour, producing predictions that cluster tightly around the 45° line.

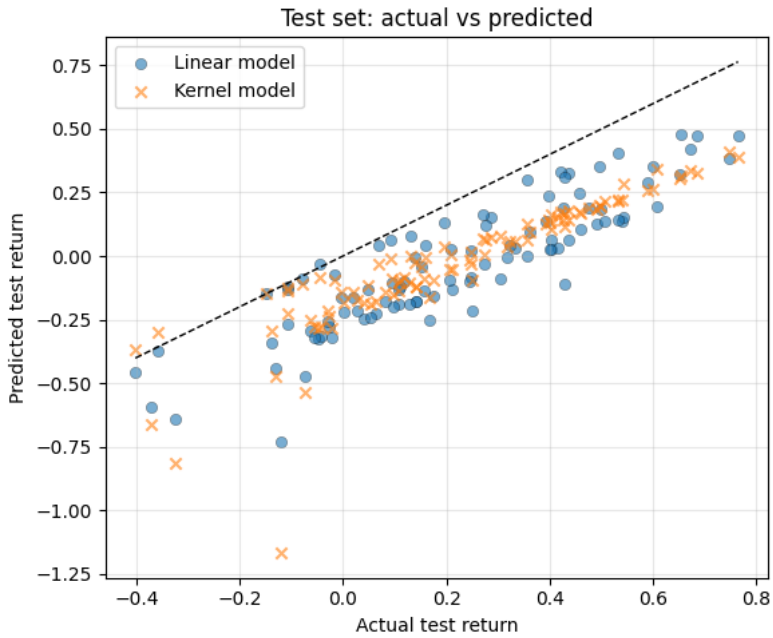

Figure 4: Test set: actual vs. predicted returns for the linear and kernel models. The kernel model forms a tighter cloud around the diagonal, although both models exhibit prediction noise due to the variability in the test sample.

On the test set (Figure 4), prediction errors increase for both models: a natural consequence of applying them to unseen data with nonlinear dynamics. The kernel model nevertheless remains slightly closer to the true values, especially in the central region of the distribution. From a financial viewpoint, these results are coherent with the simulated environment: returns depend linearly on past returns but react nonlinearly to volatility and differ across volatility regimes. A linear model with a single global slope cannot reproduce such curvature, which leads to systematic mispricing in extreme or turbulent periods. Kernel regression, by implicitly including quadratic and interaction terms, is better suited to these dynamics and improves risk discrimination. However, the limited out-of-sample gain highlights a well-known fact in quantitative finance: even sophisticated nonlinear models face severe limits when forecasting noisy asset returns.

2.6 Conclusion

Kernel regression clearly dominates the linear model in-sample, confirming the presence of nonlinear dependencies and validating the expressive power of polynomial kernels. Out-of-sample, the kernel model still performs marginally better, although both models face similar levels of noise, highlighting the challenges of forecasting returns in realistic market conditions.